WARNING Client-side evaluation is both powerful and useful but has the potential to cause problems. In general, recent versions of EF Core throw an exception if a query requires dangerous client-side evaluation, ensuring (for example) that you can't accidentally return all records to the client before filtering. For more examples, including ways to avoid these problems, see the documentation at http://mng.bz/zxP6.

At this point, you have a list of records displaying a summary of the recipe's data, so the obvious next step is loading the detail for a single record.

12.4.3 Loading a single record

For most intents and purposes, loading a single record is the same as loading a list of records. Both approaches have the same common structure you saw in figure 12.13, but when you're loading a single record, you typically use a `Where` clause that restricts the data to a single entity.

Listing 12.8 shows the code to fetch a recipe by ID, following the same basic pattern as before (figure 12.12). It uses a `Where()` LINQ expression to restrict the query to a single recipe, where `RecipeId == id`, and a `Select` clause to map to `RecipeDetailViewModel`. The `SingleOrDefaultAsync()` clause causes EF Core to generate the SQL query, execute it on the database, and build the view model.

NOTE `SingleOrDefaultAsync()` throws an exception if the previous `Where` clause returns more than one record.

Listing 12.8 Loading a single item using EF Core in `RecipeService`

```
public async Task<RecipeDetailViewModel> GetRecipeDetail(int id)   ❶
{
    return await _context.Recipes                                  ❷
```